Voyage-AI — Technical Risk Register & Prioritized Engineering Roadmap

Document Reference: `docs/reports/risks-and-roadmap.pdf` / `docs/reports/risks-and-roadmap.md`
System Version: 0.1.0 (Phase 1 Baseline)
Author: Senior Software Architect & Risk Analyst
Audit Date: October 21, 2026
Status: Implementation-Verified

---

1. Technical Risk Register

The following risk register categorizes confirmed technical risks, architectural bottlenecks, and operational constraints based on empirical evidence gathered from the codebase and host runtime inspection:

Risk ID	Risk / Issue Description	Supporting Evidence	Likelihood	Impact	Severity	Recommended Mitigation	Dependencies	Verification Criteria
RSK-01	**Host Port Collision on Port 3306**	Windows service `MySQL80` bound to `0.0.0.0:3306` (PID 8176)	High	High	**P1 (High)**	Enforce `3307:3306` port mapping in Docker Compose; document `MYSQL_PORT=3307` in `.env`	Docker Compose config	Running `docker compose up -d mysql` binds cleanly without `bind: address already in use` error
RSK-02	**External API Rate Limit & Unavailability (Open-Meteo)**	`OpenMeteoWeatherService.java` relies on public non-commercial endpoint (10,000 req/day limit)	Medium	Medium	**P1 (High)**	Implement in-memory Caffeine cache (60-minute TTL per coordinate box) to prevent duplicate lookups	Backend cache dependency	Cache hit returns identical forecast without outbound HTTP dispatch
RSK-03	**Client-Side Mock Data Divergence**	`frontend/src/App.tsx:9` queries `mockDestinations.ts`; backend has no destination endpoints	High	High	**P1 (High)**	Implement Phase 2 `DestinationController` and migrate frontend to dynamic REST API fetching	Backend Phase 2 schema	Vitest and Playwright verify destinations render from `/api/v1/destinations` payload
RSK-04	**Unauthenticated Access to Future Mutating Endpoints**	Spring Security filter chain and JWT verification not yet implemented	Low (Phase 1) / Critical (Phase 2)	High	**P1 (High)**	Implement stateless JWT Bearer filter chain (`JWT_AUTH_DESIGN.md`) before exposing mutating trip endpoints	Phase 2 implementation	MockMvc test confirms `POST /api/v1/trips` returns 401 without Bearer token
RSK-05	**AI Free-Tier Human Review & PII Leakage**	Google Gemini AI Studio free tier terms permit human review of prompts and outputs	Low	High	**P2 (Medium)**	Strip all user personal data from LLM prompts; constrain prompts strictly to internal database POI IDs (Grounded RAG)	AI Adapter implementation	Automated prompt unit tests assert regex exclusion of emails, usernames, or IPs
RSK-06	**React State Update `act(...)` Console Noise**	`npm run test:unit` outputs un-awaited state update warnings during `App.test.tsx`	High	Low	**P2 (Medium)**	Mock `fetch` synchronously in `frontend/src/setupTests.ts` or wrap tests in `waitFor()`	Frontend test setup	`npm run test:unit` completes with clean stderr and 0 warnings
RSK-07	**OSRM Public Routing Server Instability**	Public OSRM demo server has no SLA or commercial uptime guarantee	High	Medium	**P2 (Medium)**	Encapsulate OSRM behind `RoutingProvider` interface with automatic fallback to Haversine direct distance	Routing adapter design	Routing service returns Haversine estimate when OSRM connection times out
RSK-08	**OpenStreetMap Tile Usage Policy Throttling**	Leaflet client tile requests must adhere to OSM Tile Usage Policies	Medium	Low	**P3 (Low)**	Configure HTTP cache headers; provide attribution; evaluate CARTO or self-hosted vector tiles for production	Frontend map integration	Network tab confirms tiles cached locally; OSM attribution visible in footer

---

2. Prioritized Engineering Roadmap

```
+-------------------------------------------------------------------------------------+
|                                PRIORITIZED ROADMAP MATRIX                           |
+-------------------------------------------------------------------------------------+
| Priority Tier | Focus Area                                | Key Deliverables            |
|---------------|-------------------------------------------|-----------------------------|
| **P0 (Critical)**| Security & Persistent Safety           | Flyway V1, JPA entity models,|
|               |                                           | JWT authentication filters   |
| **P1 (High)** | Core Vertical Slice                       | Destination CRUD REST APIs,  |
|               |                                           | Client dynamic data binding, |
|               |                                           | Weather caching layer        |
| **P2 (Medium)**| Itinerary & Trip Management              | Multi-day trip scheduling,   |
|               |                                           | Transparent cost aggregator, |
|               |                                           | Grounded Gemini 1.5 RAG      |
| **P3 (Low)**  | Operational Polish & Production Hosting   | Interactive Leaflet maps,    |
|               |                                           | Postman collection, CI/CD,   |
|               |                                           | Cloud container deployment   |
+-------------------------------------------------------------------------------------+
```

---

3. Actionable Implementation Plan

3.1 Task 1 (P0): Relational Schema & Persistence Foundation (Phase 2 Target)

- **Problem Statement:** The database currently contains only baseline metadata (`V0__baseline.sql`). Domain travel records cannot be persisted or queried dynamically.
- **Proposed Change:** Author Flyway migration `V1__init_schema.sql` creating `users` , `destinations` , `attractions` , `accommodations` , `dining_places` , `trips` , `itinerary_days` , and `itinerary_items` with mandatory provenance columns (`source_provider` , `retrieved_at` , `verification_status`). Author corresponding JPA entities and Spring Data repositories.
- **Affected Components:** `backend/src/main/resources/db/migration/V1__init_schema.sql` , `backend/src/main/java/com/voyageai/domain/`
- **Acceptance Criteria:**
- Flyway successfully migrates schema against MySQL 8.0 on port 3307.
- JPA repositories scan and detect 6 repository interfaces.
- Repository integration tests verify CRUD operations against H2 and MySQL.

3.2 Task 2 (P0): Stateless JWT Security & Ownership Enforcement

- **Problem Statement:** Backend endpoints currently lack authentication or authorization controls.
- **Proposed Change:** Implement the architectural specification in `docs/JWT_AUTH_DESIGN.md` : author `JwtAuthenticationFilter` , `TokenProvider` (HMAC-SHA256), `AuthController` (`/api/v1/auth/login` , `/register`), and enforce user ownership in `TripService` via `findByIdAndOwnerId` .
- **Affected Components:** `backend/src/main/java/com/voyageai/security/` , `backend/src/main/java/com/voyageai/auth/`
- **Acceptance Criteria:**
- `POST /api/v1/auth/login` returns signed JWT with 60-minute expiry.
- Access to `/api/v1/trips/**` without Bearer token returns 401 Unauthorized.
- Cross-user trip mutation attempts return 404/403.

3.3 Task 3 (P1): Dynamic Discovery Catalog Integration

- **Problem Statement:** The frontend discovery grid renders static TypeScript objects (`mockDestinations.ts`) rather than live backend data.
- **Proposed Change:** Implement `DestinationController` (`GET /api/v1/destinations` with query, region, style, and budget filtering). Update `frontend/src/App.tsx` to fetch catalog data dynamically from `${apiBaseUrl}/destinations` .
- **Affected Components:** `backend/src/main/java/com/voyageai/domain/destination/` , `frontend/src/App.tsx`
- **Acceptance Criteria:**
- Client retrieves destination cards dynamically from Spring Boot backend.
- Filter state mutations trigger server-side or client-cached re-querying.
- Graceful degradation: client displays error banner if backend is unreachable.

3.4 Task 4 (P2): Grounded AI Itinerary Proposal Generator (Phase 5 Target)

- **Problem Statement:** AI itinerary generators frequently hallucinate fake places and inaccurate pricing.
- **Proposed Change:** Implement `GeminiAiService` consuming Google Gemini 1.5 Flash via AI Studio free tier. The service queries verified POIs from the local database, injects only verified entity IDs into the system prompt, and validates that generated proposals only reference authentic IDs.
- **Affected Components:** `backend/src/main/java/com/voyageai/integration/ai/`
- **Acceptance Criteria:**
- Prompts contain strictly zero user PII.
- Generated proposals only contain POI IDs existing in the local database.
- Service gracefully falls back to unassisted manual planning if Gemini rate limits (15 RPM) or times out.

---

4. Verification Milestones & Quality Gates

```
[Milestone 1: Persistence & Auth Gate]
  ├─ Criteria: Flyway V1 schema applies cleanly; JPA repositories pass integration tests
  └─ Gate: All /api/v1/trips/** endpoints reject unauthenticated requests with 401

[Milestone 2: Dynamic Vertical Slice Gate]
  ├─ Criteria: React client renders destinations fetched from /api/v1/destinations
  └─ Gate: Vitest and Playwright tests pass against live Spring Boot and MySQL containers

[Milestone 3: Grounded AI Gate]
  ├─ Criteria: Gemini AI adapter generates schema-valid proposals using verified POI IDs
  └─ Gate: Zero PII detected in prompt telemetry; rate-limit fallback tested cleanly

[Milestone 4: Production Deployment Gate]
  ├─ Criteria: Docker Compose boots clean full stack; SSL/TLS termination configured
  └─ Gate: Security scan confirms zero hardcoded secrets; Postman collection passes
```